Mettre en place une solution informatique pour l'entreprise

Module SAE 2.03

Situation d'Apprentissage et d'évaluation

Réalisé par :
Guermat Mohamed-Yacob
Baroudi Jawher

Encadrants :
Madame Graba

Université Sorbonne Paris Nord
IUT de Villetaneuse – Département Réseaux & Télécommunications

Table des matières

Introduction 2
1. Base de Donnée 3
Structure générale : 3
1. Client 3
2. PlageAdresse 3
3. StatutIP 4
2. Page de connexion 4
3. Page d'accueil 5
4. Liste des clients 6
5. Ajout d'un client 7
6. Page de détail client 8
7. Suppression d'un client 9
8. Gestion des statuts d'IP 9
9. Séparation par groupe (plages IP) 10
Conclusion 11
Difficultés rencontrées : 11
Solutions apportées : 12

Introduction

Dans le cadre de la SAE 2.03, notre mission était de mettre en place une application web permettant à une start-up de gérer efficacement l'attribution d'adresses IP publiques à ses clients. Cette plateforme devait permettre, via une interface simple et intuitive, de gérer une base client, d'automatiser la configuration réseau, et de garantir une vision globale de l'utilisation des ressources IP.

L'objectif principal était de développer un outil capable de :

- gérer l'ensemble des données clients via une base de données relationnelle,
- attribuer automatiquement des adresses IP issues de plages distinctes selon le groupe du client,
- générer automatiquement des configurations réseau (VLAN, VRF, RD),
- filtrer les données par groupe ou statut,
- et proposer un export des configurations Cisco pour intégration directe sur des équipements.

Ce projet a mobilisé nos compétences en :

- développement web (Python, Flask, HTML, CSS),
- structuration et interrogation de bases de données (SQLAlchemy + SQLite),
- et logique d'adressage IP.

1. Base de Donnée

Notre application repose sur une **base de données relationnelle** centralisant toutes les informations nécessaires à la gestion des clients, de leurs adresses IP, et de leur statut réseau.

Structure générale :

La base contient **3 tables principales** :

1. Client

Contient toutes les données liées au client :

- Informations personnelles (nom, prénom, email...)
- Adresse IP attribuée
- Informations réseau : vlan, vrf, rd
- Références vers :
  - une **plage d'adresses** (id_plage)
  - un **statut IP** (statut_ip)
- Date de création

Chaque client est automatiquement associé à :

- une IP libre dans la plage correspondant à son groupe
- un statut (libre, réservée, allouée) selon son état dans l'application

2. PlageAdresse

Contient les sous-réseaux définis par groupe :

- adresse_reseau (ex. : 164.166.1.0)
- masque (ex. : /28)
- site (ex. : Groupe 1, Groupe 2...)

Chaque **plage d'adresses** est utilisée pour attribuer automatiquement des adresses IP disponibles aux nouveaux clients selon leur groupe.

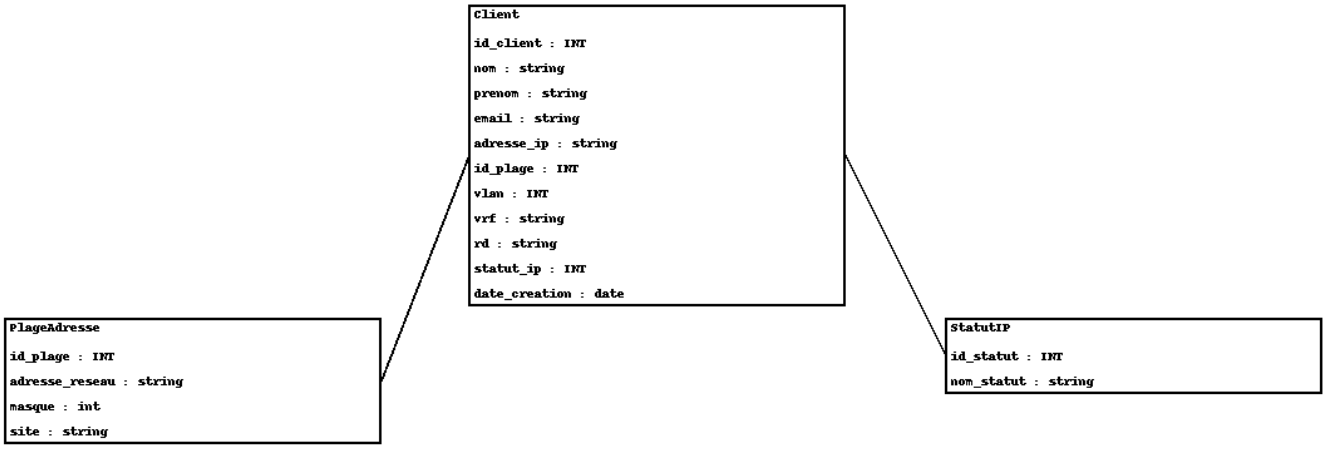
3. StatutIP

Gère les différents **états des adresses IP** :

- libre : disponible
- réservée : bloquée temporairement

- allouée : attribuée à un client actif

Ce système permet à l'administrateur de **filtrer dynamiquement** les clients et de contrôler l'utilisation des plages.



2. Page de connexion

La page de connexion est le point d'entrée sécurisé du site. L'authentification vérifie les identifiants saisis via la base de données, puis ouvre une session en cas de succès.

Fonctionnalités :

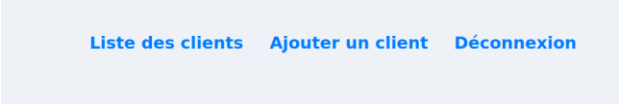
- Vérification du nom d'utilisateur et du mot de passe.
- En cas d'échec, un message d'erreur s'affiche.
- Redirection vers la page d'accueil (menu) après connexion.

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Connexion</title>
  <style>
    body { font-family: Arial; background: #f5f5f5; margin:40px; color:#333; }
    h1 { text-align:center; color:#0066cc; }
    form { max-width:400px; margin:auto; padding:20px; background:#fff;
           border-radius:8px; box-shadow:0 2px 6px rgba(0,0,0,0.1); }
    label,input,button { display:block; width:100%; margin-top:10px; padding:8px; }
    button { background:#0066cc; color:#fff; border:none; cursor:pointer; }
    button:hover { background:#004d99; }
  </style>
</head>
<body>
  <h1>Connexion</h1>
  <form method="post" action="{{ url_for('login') }}">
    <label>Nom d'utilisateur :</label>
    <input type="text" name="username" required autofocus>
    <label>Mot de passe :</label>
    <input type="password" name="password" required>
    <button type="submit">Connexion</button>
  </form>
</body>
</html>
```

3. Page d'accueil

Une fois connecté, l'utilisateur accède à un menu centralisé regroupant les principales fonctionnalités :

- Accès à la liste des clients.
- Formulaire pour ajouter un nouveau client.
- Déconnexion.
- L'objectif est de fournir un point d'entrée clair et efficace à l'administrateur.




```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}{% endblock %}</title>
  <style>
    body { font-family:Arial; background:#f2f2f2; padding:20px; color:#333; }
    nav { text-align:center; margin-bottom:20px; }
    nav a { margin:0 10px; color:#0066cc; text-decoration:none; }
    nav a:hover { text-decoration:underline; }
    .btn { padding:8px 12px; background:#0066cc; color:#fff; border:none;
           border-radius:4px; cursor:pointer; text-decoration:none; }
    .btn:hover { background:#004d99; }
  </style>
</head>
<body>
  <nav>
    <a href="{{ url_for('admin_index') }}">Liste des clients</a> |
    <a href="{{ url_for('ajout_client') }}">Ajouter un client</a> |
    <a href="{{ url_for('logout') }}">Déconnexion</a>
  </nav>
  {% block content %}{% endblock %}
</body>
</html>
```

4. Liste des clients

Cette page affiche un tableau listant tous les clients enregistrés.
Pour chacun, les données suivantes sont visibles :

- Les informations personnelles (nom, prénom, groupe, email, téléphone)
- Les données réseaux (adresse IP, VLAN, VRF, Route Distinguisher (RD), Statut IP)
- Actions possibles : voir le détail, supprimer un client, ou télécharger une configuration Cisco

Fonctionnalité de filtre :

- Rechercher par nom ou numéro client
- Filtrer par statut d'IP (allouée, libre, réservée)
- Filtrer par groupe (1, 2 ou 3)

Liste des Clients

Nom ou n° client | Tous les groupes | Tous les statuts | Filtrer

N° Client	Nom	Prénom	Groupe	Adresse IP	VLAN	VRF	RD	Statut	Actions
1	GUERMAT	Mohamed-Yacob	Groupe 1	164.166.1.1	VLAN-1	GUERMAT	65556:1	allouée	Config PE Détail Supprimer
2	Baroudi	Jawher	Groupe 1	164.166.1.17	VLAN-2	Baroudi	65556:2	allouée	Config PE Détail Supprimer
3	ALBUISSON	Jeffen	Groupe 2	164.166.2.33	VLAN-3	ALBUISSON	65556:3	allouée	Config PE Détail Supprimer
4	DHANARAJU	Mathis	Groupe 2	164.166.2.49	VLAN-4	DHANARAJU	65556:4	allouée	Config PE Détail Supprimer
5	Latafi	Haitham	Groupe 3	164.166.3.65	VLAN-5	Latafi	65556:5	allouée	Config PE Détail Supprimer
6	BELGHITRI	Billel	Groupe 3	164.166.3.81	VLAN-6	BELGHITRI	65556:6	allouée	Config PE Détail Supprimer

```html
</head>
<body>
  <div class="container">
    <nav>
      <a href="{{ url_for('admin_index') }}">Liste des clients</a>
      <a href="{{ url_for('ajout_client') }}">Ajouter un client</a>
      <a href="{{ url_for('logout') }}">Déconnexion</a>
    </nav>

    <h1>Liste des Clients</h1>

    <form class="search-form" method="get" action="{{ url_for('admin_index') }}">
      <input type="text" name="q" value="{{ q }}" placeholder="Nom ou n° client">
      <select name="site">
        <option value="">Tous les groupes</option>
        <option value="Groupe 1" {% if site=='Groupe 1' %}selected{% endif %}>Groupe 1</option>
        <option value="Groupe 2" {% if site=='Groupe 2' %}selected{% endif %}>Groupe 2</option>
        <option value="Groupe 3" {% if site=='Groupe 3' %}selected{% endif %}>Groupe 3</option>
      </select>
      <select name="statut">
        <option value="">Tous les statuts</option>
        <option value="allouée"  {% if statut=='allouée' %}selected{% endif %}>Allouée</option>
        <option value="libre"    {% if statut=='libre'   %}selected{% endif %}>Libre</option>
        <option value="réservée" {% if statut=='réservée'%}selected{% endif %}>Réservée</option>
      </select>
      <button type="submit">Filtrer</button>
    </form>

    <table>
      <thead>
        <tr>
          <th>N° Client</th><th>Nom</th><th>Prénom</th><th>Groupe</th><th>Adresse IP</th>
          <th>VLAN</th><th>VRF</th><th>RD</th><th>Statut</th><th>Actions</th>
        </tr>
      </thead>
      <tbody>
        {% for client in clients %}
        <tr>
          <td>{{ client.numero_client }}</td>
          <td>{{ client.nom }}</td><td>{{ client.prenom }}</td>
          <td>{{ client.site }}</td>
          <td>{{ client.adresse_ip }}</td>
          <td>{{ client.vlan }}</td>
          <td>{{ client.vrf }}</td>
          <td>{{ client.route_distinguisher }}</td>
          <td>{{ client.statut_ip }}</td>
          <td class="actions">
            <a class="config-link" href="{{ url_for('download_config', id=client.numero_client) }}">Config PE</a>
            <a class="detail-link" href="{{ url_for('detail_client', numero_client=client.numero_client) }}">Détail</a>
            <form action="{{ url_for('supprimer_client', numero_client=client.numero_client) }}" method="post" style="display:inline">
              <button type="submit" class="delete" onclick="return confirm('Supprimer ce client ?');">Supprimer</button>
            </form>
          </td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
</body>
</html>
```

5. Ajout d’un client

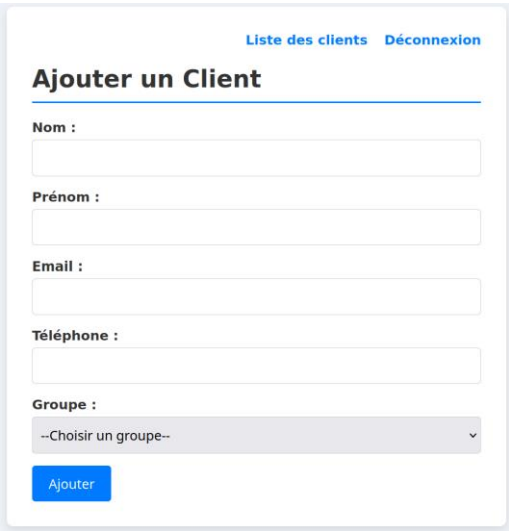
Le formulaire permet à l’administrateur d’ajouter un nouveau client avec les informations suivantes :

- Nom, prénom, téléphone, email
- Groupe (1, 2, ou 3) → détermine la plage IP
- Affectation automatique d’une IP libre dans la plage associée
- Génération automatique du VLAN, VRF et RD

L’ajout d’un client est l'une des fonctions centrales de notre application. Grâce à un formulaire simple, l’administrateur peut saisir les informations essentielles.
Dès validation, l’application :

- sélectionne automatiquement une adresse IP libre dans la plage du groupe choisi,
- génère dynamiquement les paramètres réseau associés (VLAN, VRF, RD),
- stocke toutes ces informations dans la base de données.

Ce processus garantit une cohérence d’adressage et évite tout conflit IP.




```
</head>
<body>
  <div class="container">
    <nav>
      <a href="{{ url_for('admin_index') }}">Liste des clients</a>
      <a href="{{ url_for('logout') }}">Déconnexion</a>
    </nav>

    <h1>Ajouter un Client</h1>

    <form method="post" action="{{ url_for('ajout_client') }}">
      <div class="row">
        <label for="nom">Nom :</label>
        <input type="text" id="nom" name="nom" required>
      </div>
      <div class="row">
        <label for="prenom">Prénom :</label>
        <input type="text" id="prenom" name="prenom" required>
      </div>
      <div class="row">
        <label for="email">Email :</label>
        <input type="email" id="email" name="email" required>
      </div>
      <div class="row">
        <label for="telephone">Téléphone :</label>
        <input type="text" id="telephone" name="telephone" required>
      </div>
      <div class="row">
        <label for="site">Groupe :</label>
        <select id="site" name="site" required>
          <option value="">--Choisir un groupe--</option>
          <option value="Groupe 1">Groupe 1</option>
          <option value="Groupe 2">Groupe 2</option>
          <option value="Groupe 3">Groupe 3</option>
        </select>
      </div>
      <button type="submit">Ajouter</button>
    </form>
  </div>
</body>
</html>
```

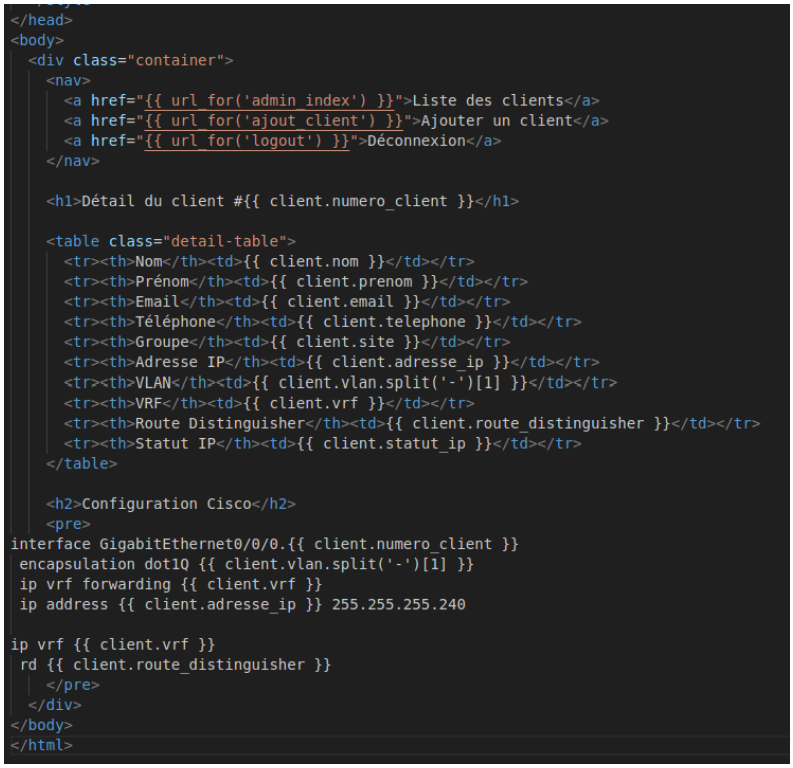
6. Page de détail client

Cette page permet de consulter les détails d’un client précis :

- Informations personnelles et réseau

- Génération automatique de la configuration Cisco à copier/coller

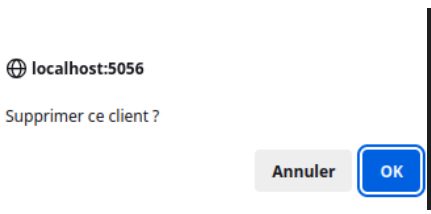
La fiche de chaque client permet une consultation détaillée de ses données personnelles et réseau. On y retrouve l'adresse IP allouée, le groupe associé, ainsi que les paramètres techniques : VLAN, VRF et Route Distinguisher.
Une zone dédiée permet de copier la configuration Cisco générée automatiquement, facilitant ainsi son déploiement sur les équipements réels.



```
</head>
<body>
  <div class="container">
    <nav>
      <a href="{{ url_for('admin_index') }}">Liste des clients</a>
      <a href="{{ url_for('ajout_client') }}">Ajouter un client</a>
      <a href="{{ url_for('logout') }}">Déconnexion</a>
    </nav>

    <h1>Détail du client #{{ client.numero_client }}</h1>

    <table class="detail-table">
      <tr><th>Nom</th><td>{{ client.nom }}</td></tr>
      <tr><th>Prénom</th><td>{{ client.prenom }}</td></tr>
      <tr><th>Email</th><td>{{ client.email }}</td></tr>
      <tr><th>Téléphone</th><td>{{ client.telephone }}</td></tr>
      <tr><th>Groupe</th><td>{{ client.site }}</td></tr>
      <tr><th>Adresse IP</th><td>{{ client.adresse_ip }}</td></tr>
      <tr><th>VLAN</th><td>{{ client.vlan.split('-')[1] }}</td></tr>
      <tr><th>VRF</th><td>{{ client.vrf }}</td></tr>
      <tr><th>Route Distinguisher</th><td>{{ client.route_distinguisher }}</td></tr>
      <tr><th>Statut IP</th><td>{{ client.statut_ip }}</td></tr>
    </table>

    <h2>Configuration Cisco</h2>
    <pre>
interface GigabitEthernet0/0/0.{{ client.numero_client }}
 encapsulation dot1Q {{ client.vlan.split('-')[1] }}
 ip vrf forwarding {{ client.vrf }}
 ip address {{ client.adresse_ip }} 255.255.255.240

ip vrf {{ client.vrf }}
 rd {{ client.route_distinguisher }}
    </pre>
  </div>
</body>
</html>
```

7. Suppression d'un client

Depuis la liste, l'administrateur peut cliquer sur un bouton pour supprimer un client. Une confirmation est demandée.
Une boîte de confirmation permet d'éviter les suppressions accidentelles.
Une fois validée, la suppression libère automatiquement l'adresse IP, qui redevient "libre" dans la base.



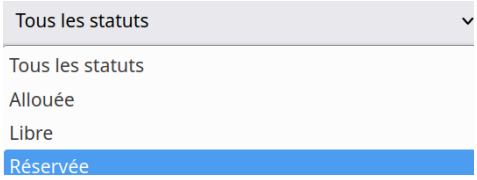

```
@app.route('/supprimer/<int:numero_client>', methods=['POST'])
def supprimer_client(numero_client):
    if session.get('username') != 'admin':
        return redirect(url_for('login'))
    c = Client.query.filter_by(numero_client=numero_client).first()
    if c:
        db.session.delete(c)
        db.session.commit()
    return redirect(url_for('admin_index'))
```

8. Gestion des statuts d'IP

Chaque adresse IP peut avoir l'un des statuts suivants :

- **Allouée** : affectée à un client actif
- **Réservée** : bloquée temporairement
- **Libre** : disponible pour de nouveaux clients

Ce statut est stocké dans la base de données et mis à jour automatiquement.
L'administrateur peut filtrer les clients en fonction de ce critère, pour mieux organiser la gestion réseau et anticiper les besoins.



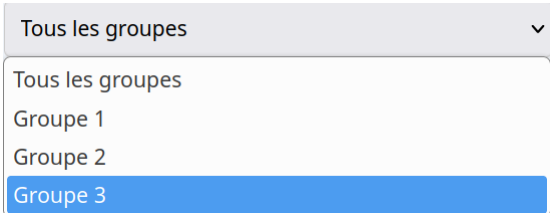

```
statut_ip = db.Column(db.String(50), default='allouée')  # allouée / réservée / libre
```

9. Séparation par groupe (plages IP)

Le projet impose une séparation des clients par groupe, chacun ayant une plage d'adresses propre :

- Groupe 1 → 164.166.1.0/28
- Groupe 2 → 164.166.2.0/28
- Groupe 3 → 164.166.3.0/28

Lorsqu'un client est ajouté, l'application sélectionne automatiquement une adresse **libre** dans la plage du groupe indiqué.
Cela garantit une organisation cohérente du plan d'adressage.




```
def get_next_ip(site):
    # Détermine la /24 de base selon le site
    base = {'Groupe 1': 1, 'Groupe 2': 2, 'Groupe 3': 3}.get(site, 3)
```

Conclusion

Ce projet nous a permis de mettre en pratique nos compétences en développement web, en structuration de base de données et en gestion d'un plan d'adressage réseau. Grâce à cette plateforme, une start-up peut désormais gérer facilement les IP publiques de ses clients tout en assurant une organisation claire par groupe et par statut. Nous avons su collaborer efficacement, en répartissant les tâches et en mettant en place une organisation claire par fichier (HTML, Python, SQL).

Difficultés rencontrées :

Plusieurs obstacles techniques ont été rencontrés au cours de la réalisation de ce projet, notamment :

- **Impossibilité d'utiliser PostgreSQL :**
  Initialement prévu pour fonctionner avec une base PostgreSQL, le projet a dû être adapté à SQLite via Flask, car l'installation et la configuration de PostgreSQL n'étaient pas possibles sur nos machines personnelles. Ce changement a nécessité des ajustements sur les requêtes SQL et la gestion des fichiers.
- **Problème d'accès à la base SQLite avec Flask :**
  Lors des premiers tests, Flask ne trouvait pas le fichier .db à cause d'un chemin incorrect. La solution a été d'utiliser des chemins relatifs dynamiques avec os.path.join() pour assurer la compatibilité sur tous les systèmes.

- **Erreurs dans l'attribution automatique d'IP :**
  Des conflits d'IP ou des sauts de blocs se produisaient. Cela venait d'une mauvaise logique dans le calcul des IPs disponibles et du statut d'allocation. Après correction, la génération se fait désormais par bloc de 16 adresses par groupe, et respecte l'ordre croissant.
- **Affichage incorrect des filtres (groupe, statut, recherche) :**
  Lors de l'application des filtres, les sélections disparaissaient après soumission du formulaire. En corrigeant la route Flask pour transmettre les variables GET au template, cela a permis de garder les champs sélectionnés visuellement.
- **Génération de configuration Cisco mal indentée :**
  Le bloc de configuration Cisco affichait un format brouillon. En utilisant les blocs <pre> HTML et la génération via template Jinja2, l'indentation a été respectée.
- **Problème de rafraîchissement après suppression de client :**
  Le tableau ne se mettait pas à jour après suppression. Le problème venait de l'absence de redirection. Résolu en utilisant redirect(url_for('admin_index')).

Solutions apportées :

Face à ces difficultés, plusieurs choix de conception ont été faits :

- Utilisation de Flask avec SQLite pour permettre le développement local simple sans dépendance serveur
- Structuration modulaire du projet (fichier app.py, liste.py, modeles.py) pour un meilleur découpage logique
- Centralisation des fonctions réutilisables comme get_next_ip() ou get_next_vlan() dans liste.py
- Génération automatique des VLAN, IP, VRF et RD pour simplifier le remplissage de formulaires
- Tests fréquents et incrémentaux à chaque ajout de fonctionnalité (ajout client, suppression, filtre...)
- Interface utilisateur accessible : les filtres sont visuellement simples à comprendre, même pour des non-techniciens, avec des menus déroulants explicites
- Fichiers de configuration Cisco générés dynamiquement en fonction des champs client, prêts à l'exportation